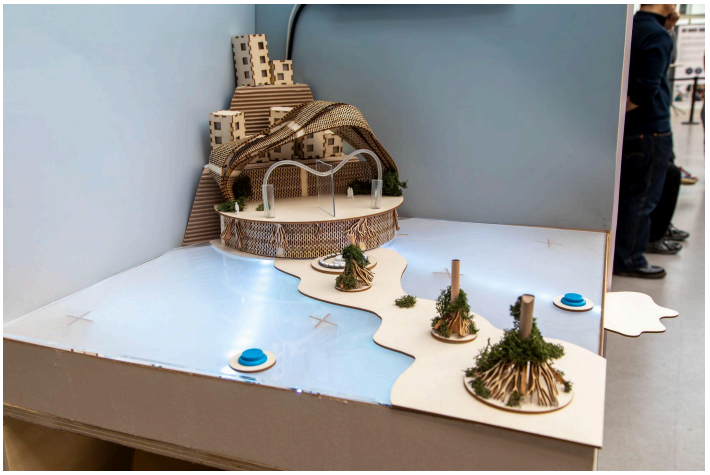
Guide technique de réalisation et d'installation du projet NDAROS



Sylvain BAUDOUIN
Roudy HAJO
Maxime LETTIER

23 Mar 2026

Table des matières :

1 Le matériel utilisé....3
1.1 Carte Arduino....3
1.2 Capteur de détection de mouvement....3
1.3 Capteur de détection d'aimant....3
1.4 Bouton à poussoir....4
1.5 Bandeau de lumière type LED....4
1.6 Alimentation....4
2 L'installation du matériel....6
2.1 Le montage des fils électriques....6
2.2 Les branchements externes....7
2.2.1 Alimentation externe....7
2.2.2 Connexion USB à la carte Arduino....8
2.2.3 Connexion à l'écran externe....8
3 Les logiciels utilisés....9
3.1. Introduction....9
3.2 Installation des logiciels....9
3.2.1 Installation de Python....9
3.2.1 Installation d'Arduino IDE et téléversement du code....9
3.3 Lancement du projet....10
3.4. Structure globale de l'architecture logicielle....10
3.4.1. Principe d'organisation....10
3.4.2. Cycle de démarrage....11
4. Gestion des états avec le design pattern State....11
4.1. Justification du choix....11
4.2. Fonctionnement dans le projet....11
4.3. Apports concrets....11
5. Serveur Flask et interface de présentation....11
5.1. Rôle du serveur Flask....11
5.2. Synchronisation côté client....12
5.3. Gestion des contenus visuels....12
6. Gestion des slides : création, modification, suppression....12
6.1. Structure de données....12
6.2. Création et modification....13
6.3. Suppression et contrôle de cohérence....13

1 Le matériel utilisé

Dans le cadre de ce projet, nous avons utilisé plusieurs types de matériels pour réaliser notre dispositif de médiation, en combinant des technologies d'entrée (capteurs, boutons) et des technologies de sortie (actionneurs, LEDs).

1.1 Carte Arduino

Dans ce projet, nous avions utilisé une carte Arduino Mega pour piloter les technologies d'entrée (capteurs, boutons, etc.) et les technologies de sortie (LEDs). C'est un microcontrôleur qui offre une connectivité étendue grâce à ses 54 broches numériques et ses 16 entrées analogiques.

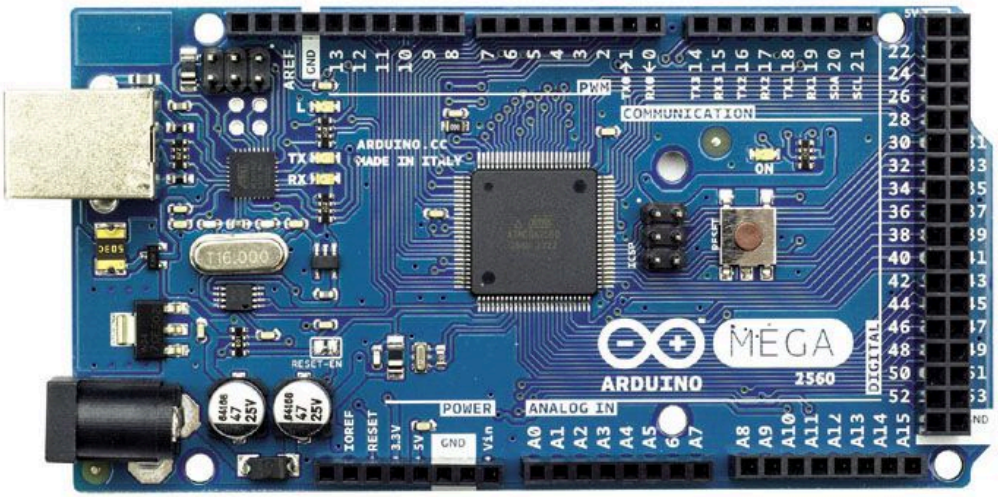


Image d'une carte Arduino Mega, source : a4.fr

1.2 Capteur de détection de mouvement

Nous avons utilisé un capteur de détection de mouvement, qui utilise la technologie infrarouge pour repérer tout déplacement devant la maquette.

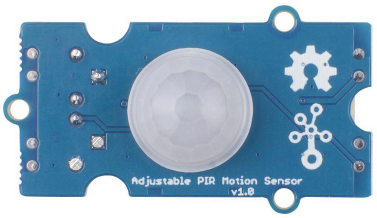


Image du capteur utilisé, source : farnell.com

1.3 Capteur de détection d'aimant

Nous avons utilisé trois capteurs de type "Effet Hall" qui permettent de détecter la présence d'un champ magnétique à proximité. Chaque capteur agit comme un interrupteur sans contact.

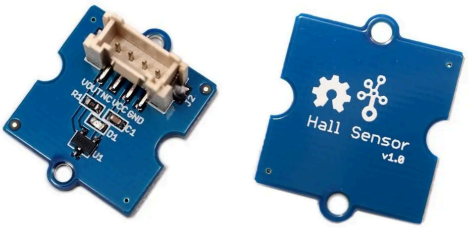


Image d'un capteur de détection d'aimant, source : hshop.vn

1.4 Bouton à poussoir

Nous avons utilisé trois boutons de type OBSF-30, ce sont des boutons qui permettent de fermer le circuit électrique par simple pression pour envoyer une commande instantanée à l'Arduino.

Exemple de boutons à poussoir que nous avons utilisés, source : plexishop.it

1.5 Bandeau de lumière type LED

Nous avons utilisé plusieurs bandeaux de lumière type LED reliés directement à l'Arduino. Il s'agit de rubans LED adressables (NeoPixel) qui permettent de contrôler chaque diode individuellement.

1.6 Alimentation

Nous avons utilisé une alimentation externe pour alimenter notre circuit d'éléments qui sont liés à l'Arduino. Nous avons utilisé 8 connecteurs WAGO pour centraliser et distribuer les lignes de puissance (5V et GND) vers tous les composants du projet.

Ces connecteurs permettent de relier proprement plusieurs fils ensemble sans soudure, assurant ainsi que chaque ruban LED et chaque capteur reçoive une tension stable, ce que les broches de l'Arduino seules ne pourraient pas supporter en raison de la consommation totale de courant.

Image d'un connecteur WAGO, source : amazon.fr

Enfin, nous avons utilisé des fils de liaison (jumpers) pour relier nos composants à l'Arduino et l'alimentation, assurant ainsi la circulation des signaux et du courant dans toute la maquette.

2 L'installation du matériel

2.1 Le montage des fils électriques

Nous avons utilisé des Wago décrits dans 1.6 Alimentation, qui nous permettent de relier tous les fils d'alimentation 5 volts et de GND. Pour ce qui est des fils reliés à l'Arduino, nous avons utilisé des longs fils.

Un composant possède généralement trois ou quatre fils :

- Fil d'alimentation (5 V) : c'est souvent un fil rouge qui permet d'apporter l'énergie électrique nécessaire au fonctionnement du composant.

- Fil de terre (GND) : c'est souvent un fil noir qui permet de fermer le circuit électrique en reliant le composant à la masse commune de l'Arduino.

- Fil de signal d'entrée (input) : c'est souvent un fil jaune qui permet de transmettre les informations captées (mouvement, aimant, bouton) vers l'Arduino.

- Fil de signal de sortie (output) : c'est souvent un fil blanc qui permet à l'Arduino d'envoyer des ordres au composant, comme l'allumage ou le changement de couleur des LEDs.

Il existe cependant une exception pour les boutons-poussoirs : ils ne possèdent que deux connecteurs. L'un est relié à une PIN numérique de l'Arduino pour envoyer le signal, tandis que l'autre est branché à la masse (GND) pour fermer le circuit lors de la pression.

L'organisation des PIN numériques

Les PIN numériques se trouvent en haut et à gauche de la carte Arduino mega, comme encadré sur l'image ci-dessous.

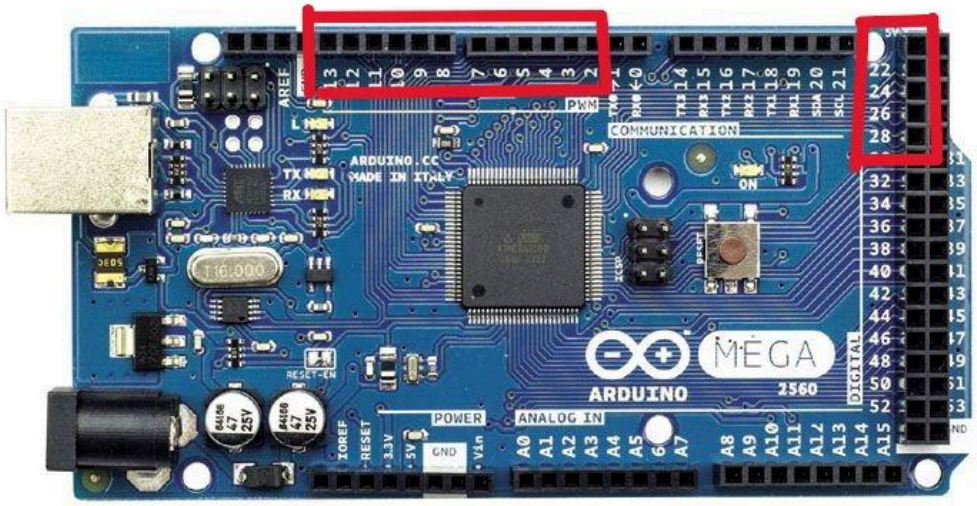


Image d'une carte Arduino Mega, source : a4.fr

Le tableau ci-dessous récapitule le câblage des différents composants sur les broches (pins) de la carte Arduino Mega correspondante.

Numéro de PIN sur la carte Arduino	Matériel connecté
2	Capteur aimant n°1
3	Capteur aimant n°2
4	Capteur aimant n°3
7	Capteur de détection de présence
8	Bouton rivière
9	Bouton océan
10	LED océan
11	LED rivière
12	LED océan principal
13	LED rivière principale
22	Bouton de la centrale
23	LED de la centrale
24	LED de la ville

2.2 Les branchements externes

2.2.1 Alimentation externe

Pour fournir l'énergie nécessaire à l'ensemble des composants, nous avons utilisé une alimentation externe d'une puissance de 50 W.

Le câblage a été réalisé selon la méthode suivante :

- Ligne positive (5 V) : nous avons relié la sortie 5 V de l'alimentation à un premier bloc de connecteurs Wago. Pour distribuer le courant à l'ensemble des éléments, plusieurs blocs Wago ont été interconnectés entre eux, créant ainsi un rail d'alimentation commun pour tous les composants du projet.

- Ligne de masse (GND) : de la même manière, un réseau de connecteurs Wago reliés en cascade regroupe tous les fils de terre (GND) des composants. Ce réseau est finalement raccordé à la borne négative (V-) de l'alimentation externe.

- Masse commune : il est impératif de relier ce réseau de Wago GND à l'une des broches GND de la carte Arduino Mega. Cette étape est cruciale pour établir une référence de tension commune, sans laquelle les signaux de commande des LEDs et des capteurs ne pourraient pas circuler correctement.

Le réseau de Wago dédié aux lignes de puissance est interconnecté par des fils rouges tandis que les blocs réservés à la masse sont reliés entre eux par des fils noirs afin de structurer proprement la distribution électrique de la maquette.

2.2.2 Connexion USB à la carte Arduino

Pour communiquer avec la carte Arduino, il faut connecter l'ordinateur à la carte avec un câble USB B (côté Arduino) vers USB A (côté ordinateur).

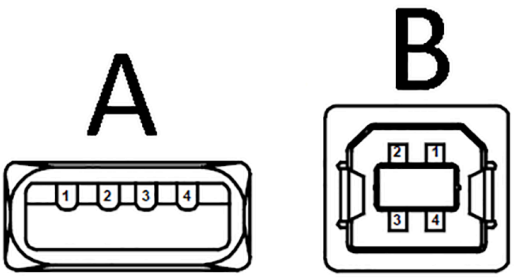


Schéma des types de câble USB, source : cable-cordon.com

2.2.3 Connexion à l'écran externe

L'affichage des diaporamas nécessite une connexion à un écran externe via un câble HDMI. Une fois le contenu ouvert dans un navigateur web, il est conseillé d'utiliser le mode de projection « Dupliquer ». Pour cela, utilisez le raccourci clavier Windows + P afin de sélectionner le mode d'affichage souhaité.

3 Les logiciels utilisés

3.1. Introduction

Dans ce projet, la carte Arduino est utilisée comme interface entre les composants physiques de la maquette (capteurs, boutons, LEDs) et la partie affichage sur écran. Afin d'assurer une restitution visuelle cohérente avec l'état réel du système, nous avons mis en place une architecture logicielle en Python, structurée autour de deux piliers :

- Une logique de contrôle basée sur le design pattern State.
- Une interface web locale basée sur Flask et Reveal.js, pilotée par un système de slides.

L'objectif global est de synchroniser, en temps réel, les interactions utilisateur et les retours capteurs avec le scénario narratif affiché.

3.2 Installation des logiciels

3.2.1 Installation de Python

Le fonctionnement du projet nécessite l'installation de Python, un environnement indispensable pour exécuter les scripts programmés dans ce langage. Pour procéder à l'installation, vous pouvez consulter le tutoriel suivant : Comment installer Python sur mon ordinateur ?

3.2.1 Installation d'Arduino IDE et téléversement du code

La carte Arduino Mega utilisée dans ce projet est déjà programmée. Néanmoins, en cas de mise à jour du code ou de remplacement de la carte, l'installation du logiciel Arduino IDE est nécessaire pour téléverser le programme. Le tutoriel suivant détaille la procédure à suivre pour télécharger le logiciel : Télécharger et installer Arduino IDE 2.0.

Le projet utilise quelques librairies Python, il faut les installer. Pour le faire, dans la barre de recherche de Windows, tapez « PowerShell ». Une fois la fenêtre ouverte, copiez-collez la commande suivante :

```
pip install pyserial requests flask pdf2image pillow
```

Pour récupérer le code source, rendez-vous sur le dépôt GitHub du projet : https://github.com/syysy/ocean.s-26 . Cliquez sur le bouton vert « <> Code », puis sur « Download ZIP ». Vous obtiendrez une archive contenant l'ensemble des projets de l'année. Après l'avoir décompressée, accédez au dossier « NDAROS », puis au sous-dossier « arduino ». Le programme principal se trouve dans le fichier nommé « main.ino ».

Une fois le logiciel installé et le programme récupéré, il suffit d'ouvrir le programme et de cliquer sur la flèche (bouton "Upload") située en haut à gauche pour téléverser le code. Avant cela, il est nécessaire de s'assurer d'avoir sélectionné la carte Arduino Mega dans la liste déroulante en haut à gauche, à côté des boutons "Verify" et "Upload".

Parfois, une erreur liée au port de communication survient. Dans ce cas, il faut se rendre dans le menu Outils (Tools), puis dans la section Port, et sélectionner le port COM correspondant à votre carte (généralement indiqué par "Arduino Mega" entre parenthèses).

3.3 Lancement du projet

Pour lancer le projet, il faut connaître le nom du port USB sur lequel est branchée la carte Arduino. Pour ce faire, il faut ouvrir l'IDE Arduino, cliquer sur le menu Outils (Tools), puis aller dans la section Port. Le nom du port utilisé (par exemple COM3 ou COM4) s'affichera alors à côté du nom de la carte Arduino Mega.

Une fois le nom du port récupéré, il est nécessaire de mettre à jour le script « main.py ». À la ligne 16, remplacez la valeur par le nom du port identifié précédemment. Dans l'exemple ci-dessous, le script est configuré pour le port « COM5 » sur Windows :

```
if sys.platform.startswith('linux') or
sys.platform.startswith('darwin'):
      PORT = '/dev/ttyACM0'
elif sys.platform == 'win32':
      PORT = 'COM5' # Ligne à mettre à jour avec votre numéro de port
(ex: COM3, COM4...)
```

Une fois la valeur du port modifiée et le fichier enregistré, le script peut être lancé selon deux méthodes différentes. La première, la plus simple, consiste à double-cliquer directement sur le fichier « main.py ». Si Python est configuré comme programme par défaut sur votre système, l'interface s'ouvrira automatiquement.

Dans le cas contraire, si le fichier s'ouvre dans un éditeur de texte (comme le Bloc-notes), il est nécessaire d'utiliser la méthode par le terminal. Pour cela, effectuez un clic droit dans un espace vide du dossier « NDAROS » au sein de l'explorateur de fichiers, puis sélectionnez l'option « Ouvrir dans le terminal » (ou « Open in Terminal »).

Il suffit alors de saisir la commande « python main.py » (ou « python3 main.py » selon votre installation) et d'appuyer sur la touche Entrée pour exécuter le programme et lancer l'interface web du projet.

3.4. Structure globale de l’architecture logicielle

3.4.1. Principe d’organisation

L’architecture est divisée en trois couches complémentaires :

1. Couche matérielle (Arduino) : acquisition des événements physiques et commande des effets lumineux.
2. Couche de décision (Python) : interprétation des événements, gestion des transitions d’états, envoi des commandes série et ordres d’affichage.

3. Couche de présentation (Flask + Reveal.js) : rendu visuel des slides et synchronisation avec l'état courant.

Cette séparation améliore la lisibilité et limite le couplage entre les responsabilités matérielles, logiques et graphiques.

3.4.2. Cycle de démarrage

Le démarrage suit une séquence contrôlée :

1. Initialisation de la liaison série avec l'Arduino, avec mécanisme de retries en cas d'échec temporaire.
2. Création du contexte d'exécution Python.
3. Lancement du serveur Flask en arrière-plan.
4. Ouverture de l'interface web en plein écran.
5. Entrée dans la boucle d'exécution pilotée par l'état initial.

Cette séquence garantit que l'interface est disponible avant les premières transitions d'état.

4. Gestion des états avec le design pattern State

4.1. Justification du choix

Le scénario de la maquette est séquentiel, mais dépend fortement d'événements externes. Sans structuration, la logique deviendrait rapidement difficile à maintenir. Le pattern State permet de modéliser chaque phase comme une entité autonome, avec ses règles d'entrée, ses conditions de sortie et ses transitions explicites.

4.2. Fonctionnement dans le projet

Le contexte conserve une référence vers l'état courant et délègue l'exécution à cet état. Chaque classe d'état représente une étape du parcours : alerte initiale, présentation, interaction sur les tuyaux, préparation de production, production, phase mangrove, fin de cycle et retour au point de départ.

Lorsqu'une condition est remplie, l'état courant déclenche une transition vers l'état suivant. Ce mécanisme rend le flux applicatif beaucoup plus clair qu'une boucle unique remplie de conditions imbriquées.

4.3. Apports concrets

Cette approche améliore la modularité, car chaque étape peut être lue et modifiée indépendamment. Elle augmente aussi la robustesse en rendant les transitions explicites et traçables. Enfin, elle facilite l'évolution du projet : l'ajout d'une étape intermédiaire ou la modification d'une temporalité se fait localement, sans réécriture complète du contrôle.

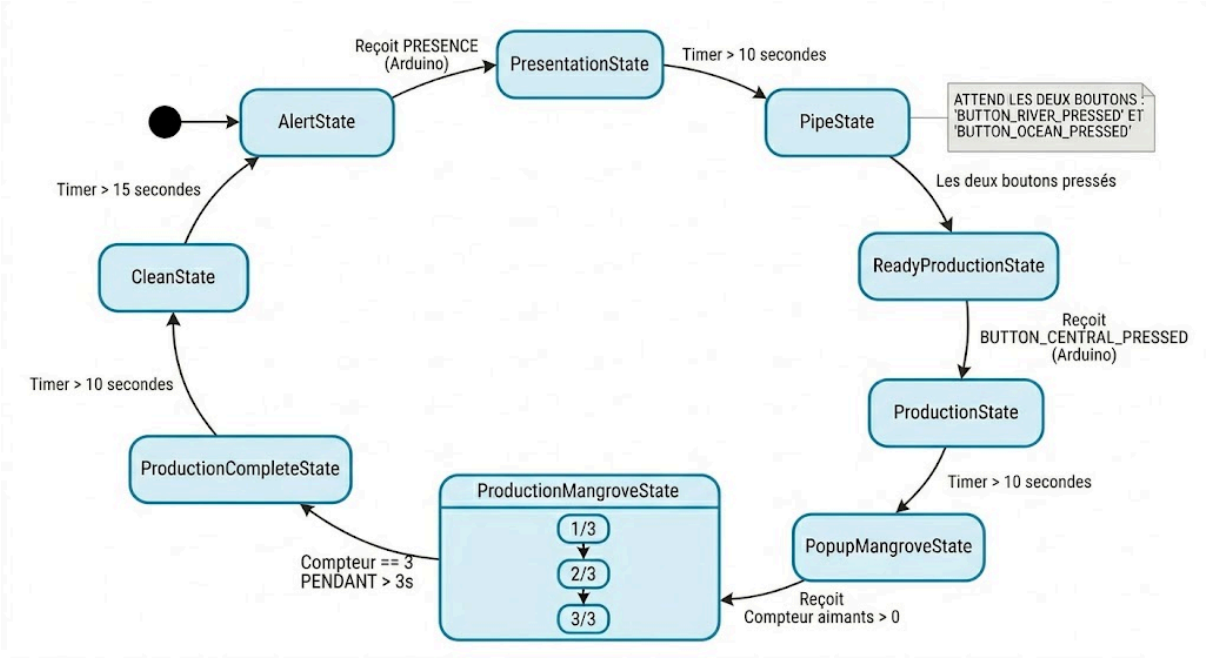


Diagramme représentant les différents états de l'application et les éléments de transitions

5. Serveur Flask et interface de présentation

5.1. Rôle du serveur Flask

Le serveur Flask fournit l'interface web et expose les points d'accès nécessaires à la navigation des slides. Il maintient un index de slide courant qui sert de référence unique pour toute l'application.

Cette centralisation est importante, car elle garantit que la machine d'états et l'interface partagent toujours la même information d'avancement.

5.2. Synchronisation côté client

Le client web, construit avec Reveal.js, charge les slides depuis l'API puis se synchronise périodiquement avec la position courante. Lorsqu'un changement est imposé par la logique Python, l'interface applique immédiatement la transition vers la slide concernée.

Le résultat est une interface stable, pilotée de façon déterministe par l'état du système.

5.3. Gestion des contenus visuels

L'application supporte des slides image, vidéo, texte et HTML. Pour les images et les vidéos, le contenu doit être stocké dans le dossier "web/static/assets".
Un mécanisme de conversion de PDF en images au démarrage simplifie l'intégration des supports graphiques produits en amont, en limitant les manipulations manuelles. Les fichiers pdf doivent être placés dans le dossier "web/assets". Si la conversion n'a pas été faite au démarrage et que la slide est toute noire lors de son affichage, vous pouvez forcer toutes les conversions avec cette ligne :

```
source ../.venv/bin/activate && python convertInPdf.py
```

Une solution plus simple est de convertir vos slides PDF en PNG sur des outils en ligne (tels que CloudConvert) pour convertir un fichier PDF en un dossier d'archive ZIP d'images PNG. Une fois la conversion faite, il faut mettre l'image dans le dossier "web/static/assets".

6. Gestion des slides : création, modification, suppression

6.1. Structure de données

Les slides sont décrites dans un fichier JSON. Chaque entrée définit le type de contenu, la ressource associée et certains paramètres d'affichage. L'ordre des éléments dans ce fichier est essentiel, car il correspond aux indices appelés par la logique d'états. Le fichier s'appelle "slides.json" dans le dossier "web/static".

```
{
 "slides": [
    {
      "type": "image",
      "src": "slide1.png",
      "title": "Titre image",
      "fullscreen": true
    },
    {
      "type": "html",
      "content": "<h1>Bienvenue</h1><p>Contenu HTML personnalisé</p>",
      "title": "Titre HTML",
      "fullscreen": false
    },
    {
      "type": "video",
      "src": "demo.mp4",
      "title": "Titre vidéo",
      "autoplay": false,
      "loop": false,
      "controls": true,
      "fullscreen": true
    }
  ]
}
```

Dans le cadre de notre projet, nos slides sont des images, notre fichier slides.json resemble à ceci :

```
{
    "slides": [
```

```
        {
            "type": "image",
            "src": "slide1.png",
            "title": "",
            "fullscreen": true
        },
        {
            "type": "image",
            "src": "slide2.png",
            "title": "",
            "fullscreen": true
        },
        {
            "type": "image",
            "src": "slide3.png",
            "title": "",
            "fullscreen": true
        },
……
```

6.2. Création et modification

Pour ajouter une nouvelle slide, il faut d'abord déposer la ressource dans les assets si nécessaire, puis ajouter une entrée JSON cohérente avec le format attendu. Pour modifier une slide, il suffit d'ajuster les champs de l'entrée existante en conservant la validité du JSON.

L'attention principale porte sur la cohérence entre le contenu éditorial et les indices utilisés dans le scénario logiciel.

6.3. Suppression et contrôle de cohérence

La suppression d'une slide implique de retirer son entrée JSON, puis de vérifier les références d'index dans la machine d'états. Sans cette vérification, un décalage peut apparaître entre la narration prévue et la slide réellement affichée.

Une validation de bout en bout reste donc indispensable après chaque évolution du jeu de slides.